KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

제어문 2
반복 제어

조건이 맞는 동안 돌고
필요하면 멈추기

```
for i in range(1, 4):
    print(i)

while stock > 0:
    stock -= 1
    if stock == 1:
        break
```

오늘의 순서

1 왜 필요한가 — 4–6쪽
몇 번 돌지 모르는 장면, for와 while 그림, 가장 짧은 while

2 range와 번호 — 7–11쪽
range(5), 시작과 끝과 간격, for i in range, range(len()), enumerate

3 while — 12–15쪽
묻기, 실행, 값 바꾸기, 재고 줄이기, 무한 반복과 멈추는 법, for와 비교

4 break와 continue — 16–20쪽
같은 주문으로 break와 continue, 점프 화살표, else로 길게 쓰기, while True

5 쌓기와 겹친 반복 — 21–26쪽
합계, 세기와 모으기, 최댓값, 구구단, 2차원 리스트, 안쪽 break

6 함정과 실습 — 27–33쪽
돌면서 지우기, 들여쓰기, range 끝, 재고, 품절, 구구단 표, 숫자 맞히기

3쪽은 노트북 준비, 34–35쪽은 막힐 때와 한 장 요약이에요.

들어가기

17에서 이어서 해요

- ✓ hello-uv 노트북, 커널 .venv — 09 · 25–27쪽
- ✓ +=, -=, // — 12
- ✓ 리스트, for, f-string, end= — 13
- ✓ 2차원 리스트 scores — 14
- ✓ 딕셔너리 items() — 15
- ✓ if, elif, else, and, or — 17

기억이 안 나면 왼쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

몇 번 돌지 모르면 for로 쓰기 어려워요

[1]
```python
money = 10000
for cup in [1, 2, 3]:
    money -= 4500
    print(f"{cup}잔째, 남은 돈 {money}원")
```

← 3번으로 정했어요

```
1잔째, 남은 돈 5500원
2잔째, 남은 돈 1000원
3잔째, 남은 돈 -3500원
```

← 돈이 모자라도 셋째 잔을 사요

몇 잔인지 모르니 횟수 대신 조건으로 말해요

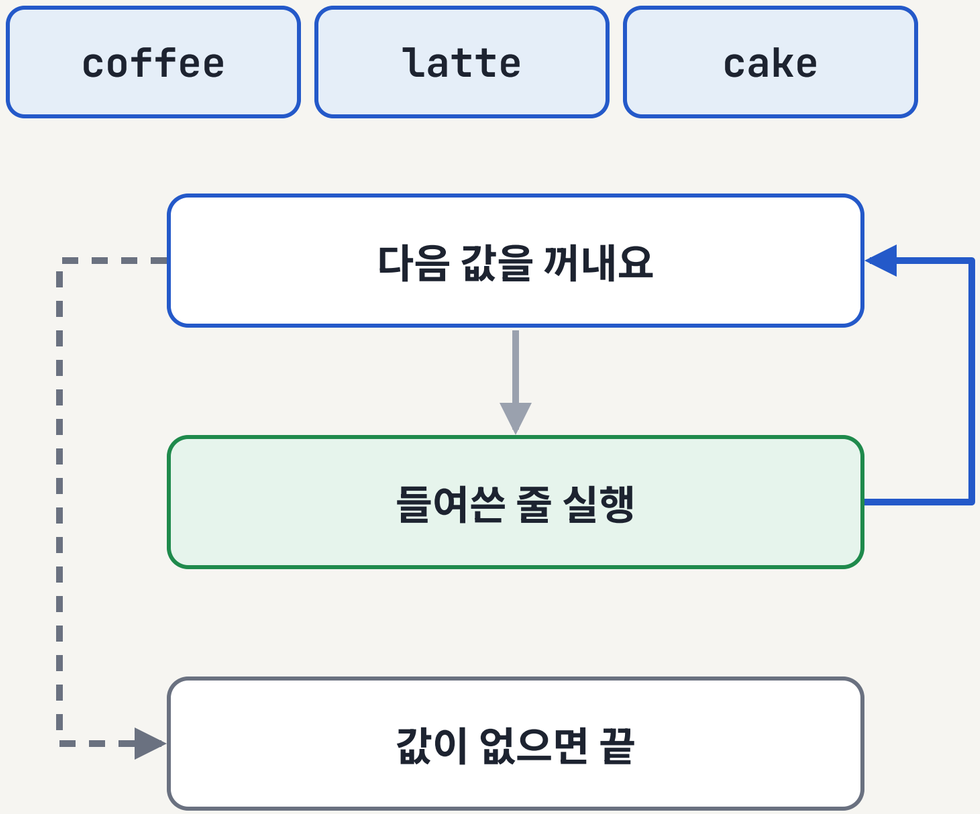
~~`for cup in [1, 2, 3]:`~~ → **돈이 4500원 이상인 동안 사요**

"~인 동안 반복"을 적는 문법이 while이에요.

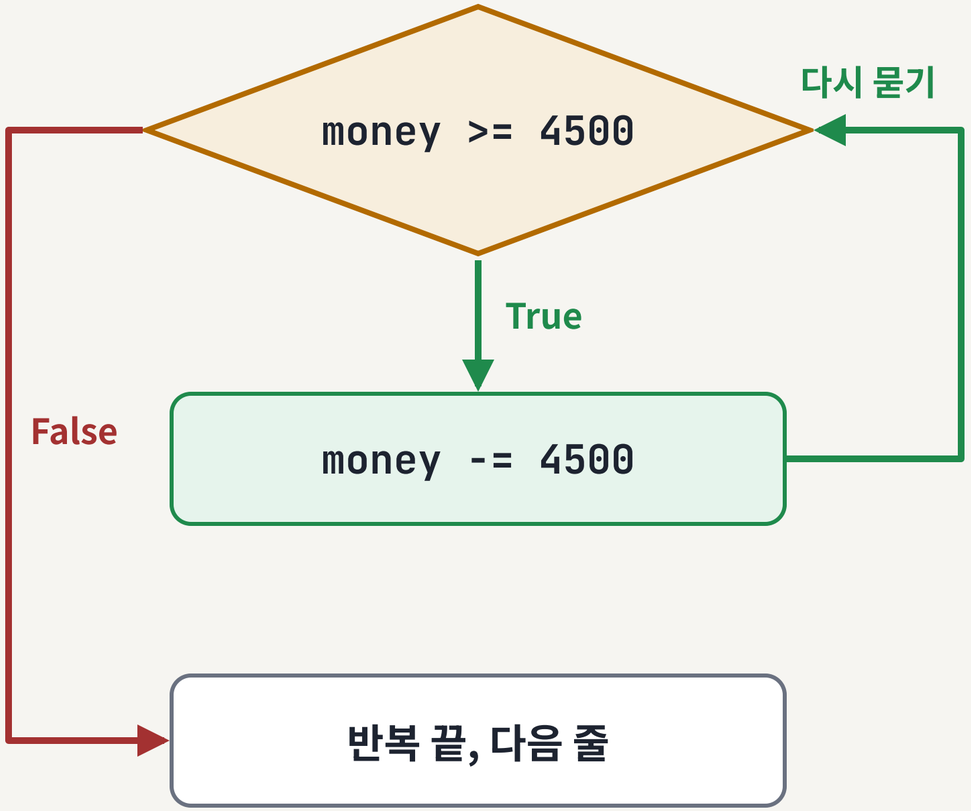
1. 왜 필요한가

for는 묶음을 하나씩, while은 조건이 True인 동안

for



while



횟수를 알면 for, 조건만 알면 while이에요.

1. 왜 필요한가

가장 짧은 while로 몇 잔인지 세요

[2]
```python
money = 10000
cups = 0
while money >= 4500:
    money -= 4500
    cups += 1
print(f"{cups}잔, 남은 돈 {money}원")
```

← 조건 뒤에 콜론, if와 같은 모양

← True인 동안 이 두 줄을 반복

```
2잔, 남은 돈 1000원
```

money	>= 4500	cups	
10000	True	0	사요
5500	True	1	사요
1000	False	2	멈춰요

while 조건: 은 조건이 True인 동안 들여쓴 줄을 반복해요.

2. range와 번호

range(5)는 0부터 4까지 다섯 개예요

[1]
```python
print(range(5))
print(list(range(5)))
print(len(range(5)))
```

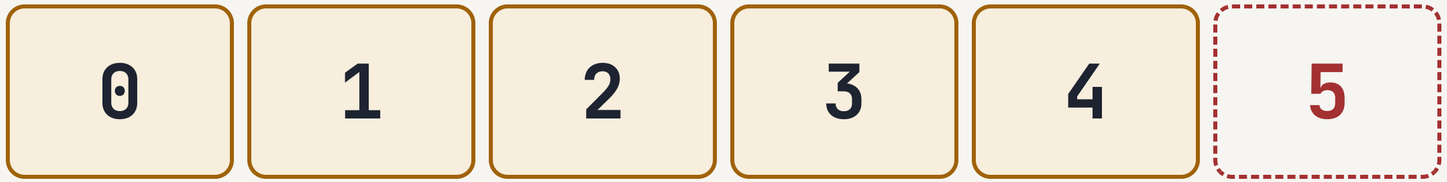
```
range(0, 5)
[0, 1, 2, 3, 4]
5
```

← 그냥 찍으면 이렇게만 나와요

← list()로 감싸면 숫자가 보여요

← 다섯 개

range(5)



← 끝 숫자 5는 안 들어가요. 13의 [0:5]와 같아요

range(n)은 0부터 n-1까지, n개예요.

2. range와 번호

시작, 끝, 간격을 정할 수 있어요

[2]
```python
print(list(range(1, 6)))
print(list(range(0, 10, 2)))
print(list(range(5, 0, -1)))
print(list(range(3, 3)))
```

```
[1, 2, 3, 4, 5]
[0, 2, 4, 6, 8]
[5, 4, 3, 2, 1]
[]
```

← 1부터 6 앞까지

← 0부터 2씩 건너뛰기

← 간격 -1이면 거꾸로, 0 앞까지

← 시작과 끝이 같으면 비어요

`range(시작, 끝, 간격)` 13의 자르기 [시작:끝:간격]과 같은 순서

끝은 늘 빠져요

1부터 5까지는 range(1, 6)이에요.

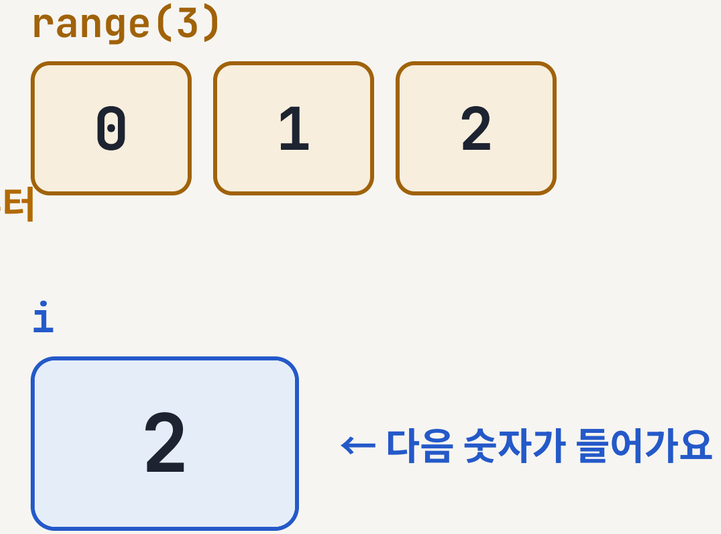
for와 range로 정한 횟수만큼 돌아요

[3]
```python
for i in range(3):
    print("주문", i)
for i in range(1, 4):
    print(f"{i}번 손님")
```

```
주문 0
주문 1
주문 2
1번 손님
2번 손님
3번 손님
```

← 리스트 자리에 range

← 사람에게 보일 번호는 1부터



"n번 반복"은 for i in range(n): 이에요.

2. range와 번호

번호와 값을 같이 쓰려면 range(len())

[4]
```python
menus = ["coffee", "latte", "cake"]
for i in range(len(menus)):
    print(i, menus[i])
```

← len(menus)는 3, 그래서 0, 1, 2
← i번 칸을 꺼내요 (13)

```
0 coffee
1 latte
2 cake
```

[5]
```python
i = 0
for m in menus:
    print(i, m)
    i += 1
```

← 번호를 직접 세는 방법, 결과가 같아요

```
0 coffee
1 latte
2 cake
```



번호가 필요하면 range(len(리스트))로 돌고 [i]로 꺼내요.

2. range와 번호

enumerate는 번호와 값을 한 쌍씩 줘요

[6]
```python
for i, m in enumerate(menus):
    print(i, m)
for i, m in enumerate(menus, 1):
    print(f"{i}. {m}")
```

← 번호는 i, 값은 m에

← 둘째 자리 1: 번호를 1부터

```
0 coffee
1 latte
2 cake
1. coffee
2. latte
3. cake
```

같은 결과, 세 가지 모양

`enumerate(menus)`	줄인 방법
`range(len(menus))`	앞 장
`i = 0 … i += 1`	직접 세기

[7]
```python
print(list(enumerate(menus)))
```

```
[(0, 'coffee'), (1, 'latte'), (2, 'cake')]
```

← (번호, 값) 튜플

번호와 값이 둘 다 필요하면 enumerate를 써요.

3. while

while은 묻기, 실행, 값 바꾸기를 돌아요

[1]
```python
count = 1
while count <= 3:
    print(f"{count}번째")
    count += 1
print("끝, count는", count)
```



← ① 시작값

← ② 묻기

← ③ 실행

← ④ 값 바꾸기

```
1번째
2번째
3번째
끝, count는 4
```

← 4 <= 3이 False라서 멈춘 뒤의 값

while에는 시작값, 조건, 값 바꾸기 세 가지가 꼭 있어야 해요.

3. while

재고가 0이 될 때까지 주문을 받아요

[2]
```python
stock = 3
while stock > 0:
    stock -= 1
    print(f"주문 받았어요. 남은 재고 {stock}")
print("품절이에요")
```

```
주문 받았어요. 남은 재고 2
주문 받았어요. 남은 재고 1
주문 받았어요. 남은 재고 0
품절이에요
```

stock	> 0	하는 일
3	True	1 빼고 찍기
2	True	1 빼고 찍기
1	True	1 빼고 찍기
0	False	반복 끝

[3]
```python
stock = 0
while stock > 0:
    stock -= 1
    print("주문 받았어요")
print("품절이에요")
```

← 처음부터 0 > 0은 False

```
품절이에요
```

← 한 번도 실행하지 않아요

조건은 반복할 때마다 처음에 물어요.

3. while

값을 안 바꾸면 끝나지 않아요

basics.ipynb ▷ Run All ■ Interrupt ↻ Restart

```python
count = 1
while count <= 3:
    print(count)
```

← ■ 정지 버튼이나 위쪽 Interrupt

← count += 1이 없어요

```
1
1
1
1
...  (1초 동안 17만 줄 넘게)
KeyboardInterrupt:
```

← count가 늘 1, 조건이 늘 True

← 멈췄다는 표시예요

터미널에서 uv run으로 돌렸다면

Windows Ctrl + C

Mac control + C

while 안에 조건을 바꾸는 줄이 있는지 먼저 봐요.

3. while

for와 while은 이럴 때 써요

[4]
```python
for i in range(1, 4):
    print(i)
i = 1
while i <= 3:
    print(i)
    i += 1
```

← 2줄

← 4줄

```
1
2
3
1
2
3
```

	for	while
언제	횟수나 묶음이 정해졌을 때	조건이 False가 될 때까지
예	메뉴 하나씩, 10번 반복	돈이 남는 동안, 재고가 있는 동안
멈춤	값이 떨어지면 저절로	값 바꾸기를 직접 적어야
실수	거의 없어요	무한 반복

둘 다 쓸 수 있으면 for가 짧고 안전해요.

4. break와 continue

break는 반복을 그 자리에서 끝내요

[1]
```python
orders = ["coffee", "cake", "latte", "tea", "cake"]
soldout = ["latte"]
for o in orders:
    if o in soldout:  ← 품절일 때만 멈추게 if 안에
        print(f"{o} 품절, 주문을 멈춰요")
        break
    print(f"{o} 주문 받았어요")
print("끝")
```

for 밖으로

```
coffee 주문 받았어요
cake 주문 받았어요
latte 품절, 주문을 멈춰요
끝
```

orders 다섯 개 중 셋만 봤어요



남은 tea, cake는 보지도 않아요

break는 "여기서 그만". 남은 차례를 버리고 반복 아래 줄로 가요.

continue는 이번 차례만 건너뛰고 계속 돌아요

[2]
```python
for o in orders:
    if o in soldout:
        print(f"{o} 품절, 건너뛰어요")
        continue
    print(f"{o} 주문 받았어요")
print("끝")
```

← break를 이 한 단어로 바꿨어요

다음 주문

```
coffee 주문 받았어요
cake 주문 받았어요
latte 품절, 건너뛰어요
tea 주문 받았어요
cake 주문 받았어요
끝
```

break · 셀 [1]



continue · 셀 [2]



continue는 "이번만 넘기기". 아래 줄을 건너뛰고 다음 차례로 가요.

4. break와 continue

화살표로 보면 어디로 가는지 보여요

[3]
```python
for i in range(1, 6):
    if i == 3:
        break
    print(i)
print("---")
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

break

continue

```
1
2
---
1
2
4
5
```

break



3에서 반복 끝. 4, 5는 돌지도 않아요

continue



3만 건너뛰고 4, 5는 계속 돌아요

break 화살표는 반복 아래로, continue 화살표는 반복 맨 위로 가요.

continue는 else로 길게 쓸 수도 있어요

[4]
```python
for o in orders:
    if o in soldout:
        print(f"{o} 품절, 건너뛰어요")
    else:
        print(f"{o} 주문 받았어요")
print("끝")
```

```
coffee 주문 받았어요
cake 주문 받았어요
latte 품절, 건너뛰어요
tea 주문 받았어요
cake 주문 받았어요
끝
```

← else 안이라 8칸

continue로 쓰면 · 셀 [2]

[2]
```python
for o in orders:
    if o in soldout:
        print(f"{o} 품절, 건너뛰어요")
        continue
    print(f"{o} 주문 받았어요")
```

주문 받기가 4칸, 한 단계 얕아요

출력은 왼쪽과 똑같아요

continue는 "이 경우는 빼고"를 맨 위에 적는 방법이에요.

4. break와 continue

while True와 break로 "될 때까지" 돌아요

[5]
```python
left = 3
n = 0
while True:
    if left == 0:
        print("재고가 떨어졌어요")
        break
    left -= 1
    n += 1
    print(f"{n}번째 주문, 남은 재고 {left}")
```

← 늘 True라 스스로는 끝나지 않아요

여기로만 나가요

```
1번째 주문, 남은 재고 2
2번째 주문, 남은 재고 1
3번째 주문, 남은 재고 0
재고가 떨어졌어요
```

← 재고가 0이 되면 나가요

while True:를 쓰면 break가 나가는 문이에요.

5. 쌓기와 겹친 반복

합계는 0에서 시작해 하나씩 더해요

[1]
```python
prices = [4500, 5000, 6500, 4500]
total = 0
for p in prices:
    total += p
print(total)
print(sum(prices))
```

```
20500
20500
```

← 원래 방법: 밖에서 0, 안에서 +=

← 13의 sum()은 위 세 줄을 대신해요

p	total += p	total
		0
4500	0 + 4500	4500
5000	4500 + 5000	9500
6500	9500 + 6500	16000
4500	16000 + 4500	20500

쌓을 변수를 반복 밖에서 0으로 두고, 안에서 +=로 더해요.

5. 쌓기와 겹친 반복

조건에 맞는 것을 세고 모아요

4500	5000	6500	4500

← 5000 이상은 두 개

[2]
```python
n = 0
for p in prices:
    if p >= 5000:
        n += 1   ← 세기
print(n)
print(prices.count(5000))
```

```
2
1  ← count()는 5000과 같은 값만 세요
```

[3]
```python
big = []
for p in prices:
    if p >= 5000:
        big.append(p)   ← 모으기
print(big)
print(len(big))
```

```
[5000, 6500]   ← 17의 골라 담기
2  ← 모은 리스트의 len이 개수
```

세기는 n += 1, 모으기는 append예요. 둘 다 if 안에 둬요.

5. 쌓기와 겹친 반복

가장 큰 값은 지금까지 1등과 비교해요

[4]
```python
big = prices[0]
for p in prices:
    if p > big:
        big = p
print(big)
print(max(prices), min(prices))
```

← 0이 아니라 첫 값으로 시작

← 더 크면 바꾸기

← 13의 max()와 min()이 한 줄로

```
6500
6500 4500
```

p	p > big	big
		4500
4500	False	4500
5000	True	5000
6500	True	6500
4500	False	6500

지금까지 가장 큰 값을 big에 두고, 더 큰 값을 만나면 바꿔요.

5. 쌓기와 겹친 반복

반복 안에 반복을 넣으면 표를 만들어요

[5]
```python
for dan in range(2, 4):
    for i in range(1, 4):
        print(f"{dan} x {i} = {dan * i}")
    print("---")
```

← 바깥 · dan = 2, 3

← 안쪽 · i = 1, 2, 3

← 4칸, 단마다 한 번

```
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
---
```

바깥이 한 번 돌 때 안쪽은 세 번

	0	1	2
0	2x1	2x2	2x3
1	3x1	3x2	3x3

바깥 2번 × 안쪽 3번 = 모두 6번 실행해요.

5. 쌓기와 겹친 반복

14의 성적표를 행과 열 번호로 돌아요

[6]
```python
scores = [[90, 85, 77],
          [70, 95, 88],
          [60, 75, 99]]
for i in range(len(scores)):
    for j in range(len(scores[i])):
        if scores[i][j] < 70:
            print(f"{i}행 {j}열: {scores[i][j]}점")
```

```
2행 0열: 60점
```

← scores[2][0], 14의 [행][열] 순서

← 바깥 i는 행 번호

← 안쪽 j는 열 번호

	0	1	2
0	90	85	77
1	70	95	88
2	60	75	99

2차원 리스트는 바깥 for가 행, 안쪽 for가 열이에요.

5. 쌓기와 겹친 반복

안쪽 break는 안쪽 반복만 멈춰요

[7]
```python
for row in scores:
    for s in row:
        if s < 80:
            print("80점 아래", s, "찾음")
            break
    print("이 행 확인 끝", row)
```

← 안쪽 for만 끝

← 바깥 for는 다음 행으로

```
80점 아래 77 찾음
이 행 확인 끝 [90, 85, 77]
80점 아래 70 찾음
이 행 확인 끝 [70, 95, 88]
80점 아래 60 찾음
이 행 확인 끝 [60, 75, 99]
```

빨강은 찾은 값, 회색은 안 본 값

	0	1	2
0	90	85	77
1	70	95	88
2	60	75	99

행마다 처음 찾은 값에서 멈추고, 다음 행은 계속 확인해요.

6. 함정과 실습

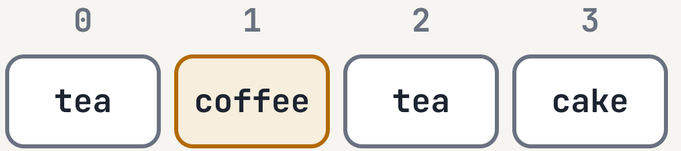
돌면서 같은 리스트를 지우면 하나를 놓쳐요

[1]
```python
orders = ["tea", "tea", "coffee", "tea", "cake"]
for o in orders:
    if o == "tea":
        orders.remove(o)
print(orders)
```

```
['coffee', 'tea', 'cake']
```
← tea가 하나 남았어요

0번 tea를 지우면 뒤가 한 칸씩 당겨져요





for는 다음 차례인 1번 coffee로 가서 당겨진 tea를 못 봐요

[2]
```python
orders = ["tea", "tea", "coffee", "tea", "cake"]
keep = []
for o in orders:
    if o != "tea":
        keep.append(o)   ← 남길 것만 새 리스트에 (17의 골라 담기)
print(keep)
```

```
['coffee', 'cake']
```

돌고 있는 리스트는 지우지 않아요. 새 리스트에 모아요.

6. 함정과 실습

들여쓰기 한 단계 차이로 결과가 달라져요

[3]
```python
total = 0
for p in [4500, 5000, 6500]:
    total += p
    print(total)
```

```
4500
9500
16000
```

[4]
```python
total = 0
for p in [4500, 5000, 6500]:
    total += p
print(total)
```

```
16000
```

[5]
```python
for p in [4500, 5000, 6500]:
    total = 0
    total += p
print(total)
```

```
6500
```

print가 for 안

돌 때마다 찍어요

print가 for 밖

다 돈 뒤 한 번

total = 0이 for 안

돌 때마다 0으로 돌아가요

쌓을 변수는 for 위에서 0으로, 결과는 for 아래에서 찍어요.

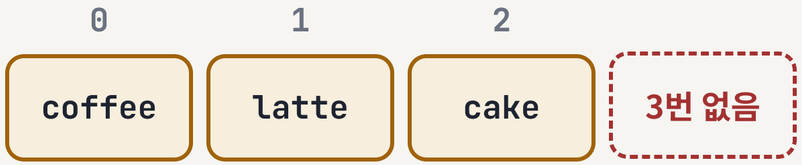
range(len())에 1을 더하면 칸을 넘어가요

[6]
```python
menus = ["coffee", "latte", "cake"]
for i in range(len(menus) + 1):
    print(menus[i])
```

← 0, 1, 2, 3

```
coffee
latte
cake
IndexError: list index out of range
```

← 없는 3번 칸



[7]
```python
print(list(range(1, 5)))
print(list(range(len(menus))))
```

```
[1, 2, 3, 4]
[0, 1, 2]
```

← 1부터 5까지는 range(1, 6)

← 딱 맞는 번호

IndexError가 나면 range의 끝 숫자부터 봐요.

6. 함정과 실습

재고가 모자랄 때까지 주문을 받아요

[8]
```python
stock = 5
orders = [2, 1, 3, 1]
sold = 0
i = 0
while i < len(orders):
    want = orders[i]
    if want > stock:
        print(f"{i + 1}번 손님 {want}잔: 재고가 {stock}잔뿐이에요")
        break
    stock -= want
    sold += want
    print(f"{i + 1}번 손님 {want}잔, 남은 재고 {stock}")
    i += 1   ← 다음 손님
print(f"판매 {sold}잔, 남은 재고 {stock}")
```

Python

```
1번 손님 2잔, 남은 재고 3
2번 손님 1잔, 남은 재고 2
3번 손님 3잔: 재고가 2잔뿐이에요
판매 3잔, 남은 재고 2
```

← 손님이 남은 동안

← 재고가 모자라면 멈춤

손님	want	stock	
1번	2	5→3	팔아요
2번	1	3→2	팔아요
3번	3	2	break

멈출 조건이 두 개면 하나는 while에, 하나는 if와 break로 적어요.

6. 함정과 실습

처음 품절이 나오면 주문을 멈춰요

[9]
```python
stock = {"coffee": 2, "latte": 1, "cake": 3}
orders = ["coffee", "latte", "coffee", "latte", "cake"]
for o in orders:
    if stock[o] == 0:
        print(f"{o} 품절이에요. 주문을 멈춰요")
        break
    stock[o] -= 1
    print(f"{o} 나갔어요. 남은 수 {stock[o]}")
print(stock)
```

← 품절이면 break

← 15처럼 값 바꾸기

```
coffee 나갔어요. 남은 수 1
latte 나갔어요. 남은 수 0
coffee 나갔어요. 남은 수 0
latte 품절이에요. 주문을 멈춰요
{'coffee': 0, 'latte': 0, 'cake': 3}
```

다섯째 cake는 재고가 있어도 안 봐요



break를 continue로 바꾸면 품절만 건너뛰고 cake도 받아요.

6. 함정과 실습

구구단 표를 한 번에 찍어요

[10]

```python
for dan in range(2, 10):
    print(f"{dan}단", end=" ")
    for i in range(1, 10):
        print(f"{dan * i:>3}", end="")
    print()
```

← end=" "로 줄을 안 바꿔요 (13)

← >3은 3칸 오른쪽 맞춤 (13)

← print()는 줄만 바꿔요

```
2단   2  4  6  8 10 12 14 16 18
3단   3  6  9 12 15 18 21 24 27
4단   4  8 12 16 20 24 28 32 36
5단   5 10 15 20 25 30 35 40 45
6단   6 12 18 24 30 36 42 48 54
7단   7 14 21 28 35 42 49 56 63
8단   8 16 24 32 40 48 56 64 72
9단   9 18 27 36 45 54 63 72 81
```

← 안쪽 for가 채운 한 줄

안쪽 for는 한 줄을, 바깥 for는 줄 수를 맡아요.

6. 함정과 실습

숫자 맞히기, 맞히면 멈춰요

[11]

```python
answer = 37
guesses = [50, 25, 37, 40]
tries = 0
for g in guesses:
    tries += 1
    if g == answer:
        print(f"{g}: 정답이에요. {tries}번 만에 맞혔어요")
        break
    elif g > answer:
        print(f"{g}: 더 작아요")
    else:
        print(f"{g}: 더 커요")
```

Python

```
50: 더 작아요
25: 더 커요
37: 정답이에요. 3번 만에 맞혔어요
```

← 맞히면 break

← 17의 elif로 힌트

넷째 추측 40은 보지 않아요



guesses를 바꾸면 끝까지 못 맞히고 힌트만 찍혀요.

정리

막히면 오류의 마지막 줄을 봐요

	이럴 때	까닭	이렇게
1	**셀이 끝나지 않고 출력이 계속 나와요**	while 조건이 늘 True예요	**■ 또는 Interrupt, 값 바꾸기 줄 넣기**
2	`KeyboardInterrupt`	직접 멈춘 표시라 오류가 아니에요	**값 바꾸기 줄 넣기**
3	`IndexError: list index out of range`	range(len(xs) + 1)처럼 칸을 넘었어요	`range(len(xs))`
4	**1부터 5까지 도는데 4에서 끝나요**	range의 끝 숫자는 빠져요	`range(1, 6)`
5	**지웠는데 하나가 남아요**	돌고 있는 리스트를 remove했어요	**새 리스트에 append**
6	**합계가 마지막 값만 나와요**	total = 0을 for 안에 넣었어요	**for 위로 옮기기**
7	**바깥 반복이 멈추지 않아요**	break는 가장 가까운 반복 하나만 끝내요	**어느 for 안인지 보기**

결과가 이상하면 반복 안에 print를 넣어 값이 어떻게 바뀌는지 찍어 봐요.

한 장 요약

`range(1, 6)`	1부터 5까지, 끝은 빠져요
`for i in range(n):`	n번 반복
`enumerate(xs, 1)`	번호와 값을 같이
while 조건:	True인 동안 반복

`break`	반복을 바로 끝내기
`continue`	이번 차례만 넘기기
`total += p`	0에서 시작해 쌓기
for 안의 for	표 만들기, 2차원 리스트

횟수를 알면 for, 조건만 알면 while이에요.

while에는 시작값, 조건, 값 바꾸기 세 가지가 꼭 있어야 해요.

다음 챕터 19에서는 여러 줄을 함수로 묶고 이름을 붙여요.